PRADITA University

IT40713 · Cloud-Native & Microservices Architecture

Container, Kubernetes dan Serverless

2 November 2026

Alfa Yohannis



Isi Pertemuan

PRADITA University

1. Tujuan dan Konteks
2. VM dan Container
3. Kubernetes
4. Managed Kubernetes dan PaaS
5. Serverless
6. Memilih Runtime
7. Aktivitas
8. Ringkasan dan Latihan

Mengapa aplikasi yang lancar di Jakarta bisa gagal di Bandung?

PRADITA University

Tujuan dan Konteks

Tujuan Pembelajaran

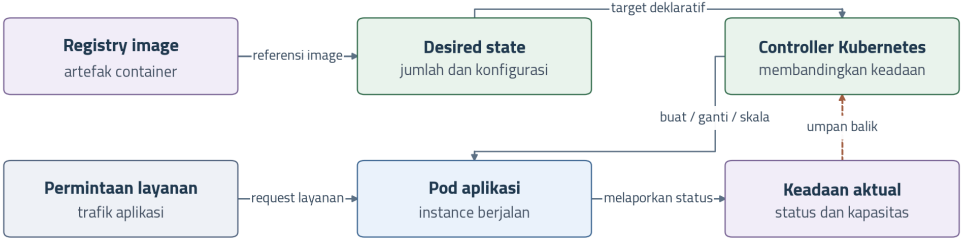
PRADITA University

- Menjelaskan perbedaan *virtual machine*, *container*, dan *serverless* dengan analogi dan contoh.
- Menjelaskan cara Kubernetes menjaga sistem tetap sesuai keadaan yang diinginkan.
- Memilih *runtime* untuk setiap kandidat layanan berdasarkan kriteria yang eksplisit.

Diagram Konsep Bab 8

IT40713 / BAB 08 / DIAGRAM KONSEP

Kubernetes merekonsiliasi keadaan yang diinginkan



Kubernetes adalah salah satu opsi runtime. Managed control plane tetap menyisakan pekerjaan aplikasi, data, dan kebijakan.

PRADITA University

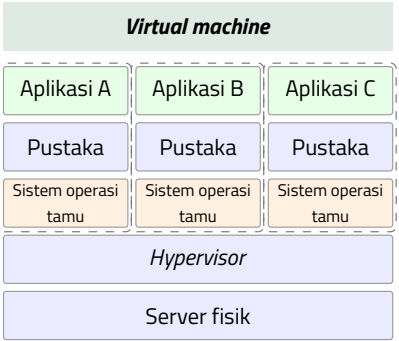
Virtual Machine dan Container

Virtual Machine dan Container

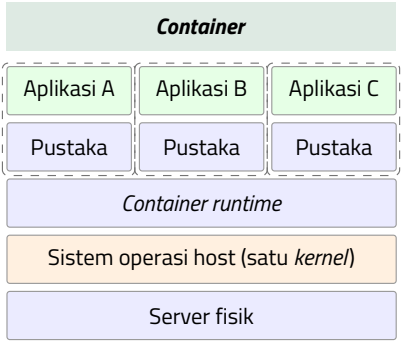
PRADITA University

- VM: komputer tiruan di atas *hypervisor*, membawa sistem operasi tamu sendiri.
- *Container*: aplikasi dan pustakanya, berjalan sebagai proses terisolasi, berbagi satu *kernel* host.
- *Container* lebih kecil dan cepat menyala; VM lebih berat, isolasinya lebih tebal.
- Analogi: peti kemas standar. Isinya berbeda, cara mengangkutnya sama.

Susunan Lapisan VM dan Container



Setiap VM membawa sistem operasi sendiri: lebih berat, lebih lambat menyala, isolasinya lebih kuat



Semua *container* berbagi satu *kernel*: ringan, menyala dalam hitungan detik, isolasinya lebih tipis

Dua Contoh dari Medikara Health

- Modul pendaftaran dikemas sebagai *container* bersama pustaka yang sudah diuji; kemasan sama dipakai di lima rumah sakit.
- Misalkan rumah sakit Depok menjalankan 12 aplikasi kecil, satu VM per aplikasi.
- Sekitar 2 GB memori per sistem operasi tamu: 24 GB habis hanya untuk sistem operasi.
- Sebagai *container* di dua host: sistem operasi yang dirawat tinggal dua.

Container Image dan Registry

Istilah	Arti	Contoh
Container image	Paket baca-saja, aplikasi dan dependensinya	`jadwal-dokter:1.4.2`
Container	*Image* yang sedang berjalan	Tiga *container* di rumah sakit Bandung
Registry	Gudang *image*	*Registry* privat di *region* Jakarta
Tag dan *digest*	Nama versi dan sidik jari isi	Produksi memakai *digest* yang sudah diuji

Format *image* distandarkan Open Container Initiative. *Registry* menjadi aset kritis, dan *image* tidak boleh memuat data pasien.

Batas Isolasi Container

Aspek	***Virtual machine***	***Container***
Batas isolasi	*Kernel* sendiri	Berbagi satu *kernel*
Waktu menyala	Lebih lama	Cepat
Cocok untuk	Aplikasi lama, isolasi ketat	Layanan baru yang sering diperbarui

- NIST SP 800-190: kelompokkan *container* menurut tujuan dan sensitivitas.
- Medikara: *container* Medikara+ tidak satu host dengan pengolah rekam medis.

PRADITA University

Orkestrasi Container dan Kubernetes

Mengapa Perlu Orkestrasi

- Misalkan 9 layanan, 3 replika, 3 lingkungan: 81 *container*.
- Server mana yang masih punya ruang?
- Siapa menyalakan ulang *container* yang mati pukul 02.00?
- Bagaimana memasang versi baru tanpa menghentikan layanan?
- Siapa menambah kapasitas pukul 07.00?

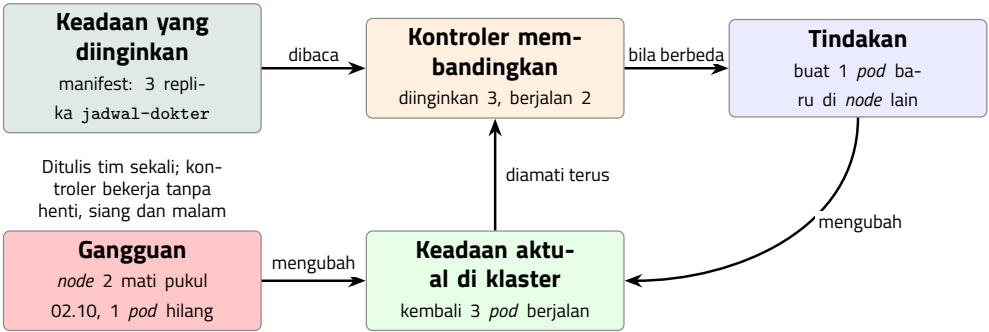
Kubernetes: orkestrator *open source*, lahir dari pengalaman Google dengan Borg dan Omega.

Desired State dan Rekonsiliasi

PRADITA University

- Tim menulis keadaan yang diinginkan di manifest: ”tiga replika versi 1.4.2”.
- Kontroler: lingkaran kendali yang mengamati, membandingkan, lalu bertindak.
- Analogi: kepala ruang perawat menjaga angka ”enam perawat per sif”.
- Konsekuensi 1: keadaan sistem dapat diaudit dari manifest.
- Konsekuensi 2: perubahan manual di klaster dikembalikan kontroler.

Satu Putaran Rekonsiliasi



Pod, Deployment, dan Service

PRADITA University

Konsep	Fungsi	Contoh Medikara
Klaster dan *node*	Sekumpulan server yang dikelola sebagai satu	Enam *node* di *region* Jakarta
Control plane	Menyimpan keadaan, menjadwalkan, menjalankan kontroler	Diurus penyedia pada *managed* Kubernetes
Pod	Unit terkecil, satu atau beberapa *container*	Satu *pod* jadwal dokter
Deployment	Jumlah replika dan versi *image*	Tiga replika `jadwal-dokter:1.4.2`
Service	Alamat tetap untuk *pod* yang berganti	Dipanggil layanan janji temu

Self-Healing dan Autoscaling

PRADITA University

- *Self-healing*: *container* gagal dinyalakan ulang, *pod* di *node* mati diganti.
- Misalkan *node* mati pukul 02.10: *pod* pengganti muncul dalam sekitar satu menit.
- *Autoscaling*: replika ditambah atau dikurangi menurut beban.
- Janji temu: 10 menjadi 50 permintaan per detik pukul 07.00 sampai 09.00, termasuk pengecekan jadwal dan kuota.
- Target prosesor 70 persen, tiga sampai delapan replika.

Membaca Survei CNCF 2025

Indikator	Angka	Cara membaca
Responden dan waktu survei	628, September 2025	Sampel komunitas, bukan sensus
Pengguna *container* yang menjalankan Kubernetes di produksi	82 persen (2023: 66 persen)	Penyebutnya pengguna *container*
Organisasi dengan model AI generatif sendiri yang memakai Kubernetes untuk inferensi	66 persen	Penyebutnya organisasi yang menjalankan model sendiri

Ekosistemnya besar, tetapi angka adopsi tidak menjawab apakah Kubernetes tepat untuk tim kecil.

Managed Kubernetes dan PaaS

Managed Kubernetes dan PaaS

PRADITA University

- *Managed* Kubernetes: penyedia menjalankan *control plane*.
- Pelanggan tetap mengurus manifest, *image*, izin akses, jaringan, dan pembaruan *node*.
- PaaS: serahkan kode atau *image*; platform mengurus server, *runtime*, dan skala.
- *Managed* Kubernetes memperlihatkan Kubernetes; PaaS menyembunyikannya.
- Analogi: dapur sendiri, sewa dapur komersial, katering.

Pembagian Tanggung Jawab

Yang dikelola	**VM (IaaS)**	**Kubernetes sendiri**	***Managed* Kubernetes**	**PaaS**	***Serverless***	**SaaS**
Data dan hak akses pengguna	Pelanggan	Pelanggan	Pelanggan	Pelanggan	Pelanggan	Pelanggan
Kode aplikasi	Pelanggan	Pelanggan	Pelanggan	Pelanggan	Pelanggan	Penyedia
Penambahan kapasitas	Pelanggan	Pelanggan	Bersama	Bersama	Penyedia	Penyedia
Runtime bahasa dan pustaka	Pelanggan	Pelanggan	Pelanggan	Bersama	Bersama	Penyedia
Orkestrasi (*control plane*)	Pelanggan	Pelanggan	Penyedia	Penyedia	Penyedia	Penyedia
Sistem operasi dan *patch*	Pelanggan	Pelanggan	Bersama	Penyedia	Penyedia	Penyedia
Server fisik dan gedung	Penyedia	Penyedia	Penyedia	Penyedia	Penyedia	Penyedia



Kendali lebih besar, beban operasi lebih berat

Lebih sedikit yang dikelola, ketergantungan pada penyedia lebih besar

Biaya Control Plane dan Irama Pembaruan

PRADITA University

- AWS: 0,10 dolar AS per klaster per jam (dukungan standar), 0,60 dolar AS (diperpanjang).
- Tiga klaster: 219 dolar AS per bulan, sekitar Rp3,6 juta (kurs asumsi Rp16.500).
- Terlambat memperbarui: 1.314 dolar AS, sekitar Rp21,7 juta per bulan.
- Kubernetes: sekitar tiga rilis minor per tahun, dukungan *patch* sekitar satu tahun.
- Misalkan tiga pembaruan per klaster per tahun: 90 hari kerja.

Kasus Medikara: Biaya Tiga Opsi

Komponen (juta rupiah per tahun)	Sendiri	*Managed*	PaaS
Control plane	180	43	0
Kapasitas komputasi sembilan layanan	600	600	720
Tenaga kerja platform (jumlah orang)	1.560 (4)	780 (2)	390 (1)
Pelatihan, alat pemantauan, dan keamanan	270	240	90
Total satu tahun	**2.610**	**1.663**	**1.200**

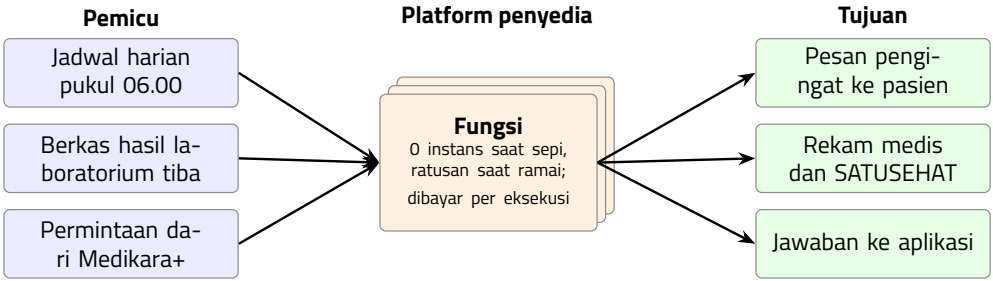
- Tenaga kerja 46,9 persen biaya *managed*; *control plane* hanya 2,6 persen.
- Usulan: PaaS untuk layanan baru pertama, hemat Rp463 juta (27,8 persen).

Serverless dan Function as a Service

Serverless dan FaaS

- Server diurus penuh penyedia; kapasitas otomatis, menyusut sampai nol.
- Tagihan mengikuti pemakaian nyata, per eksekusi.
- FaaS: fungsi kecil yang dijalankan setiap kali ada pemicu.
- Kajian Berkeley: *serverless* adalah FaaS ditambah *Backend as a Service*.
- Analogi: lampu sensor gerak di lorong kantor.

Alur Function as a Service



Biaya per Eksekusi dan Keterbatasan

PRADITA University

- AWS Lambda: 0,20 dolar AS per satu juta permintaan dan 0,0000166667 dolar AS per GB-detik (x86, AS Timur).
- Kuota gratis per bulan: satu juta permintaan dan 400.000 GB-detik.
- *Cold start*: permintaan pertama setelah sepi menunggu lebih lama.
- Batas waktu: AWS Lambda paling lama 900 detik (15 menit).
- Ketergantungan: format pemicu, *event*, dan izin akses khas penyedia.

Pola Beban yang Cocok

Pola beban	Contoh Medikara	Kecocokan
Jarang, tidak menentu	Ekspor data untuk auditor	Sangat cocok
Berkala terjadwal	Pengingat kontrol 06.00 dan 18.00	Cocok
Dipicu kedatangan data	Hasil laboratorium tiap 30 menit	Cocok, dengan syarat
Lonjakan tajam singkat	Pendaftaran kampanye vaksinasi	Cocok, dengan catatan
Tinggi dan stabil 24 jam	Antarmuka SIMRS rawat inap	Kurang cocok
Proses panjang	Rekonsiliasi klaim bulanan tiga jam	Tidak cocok tanpa dipecah

Kasus Medikara: Pengingat Jadwal Kontrol

PRADITA University

- Misalkan 40 persen dari 1,2 juta kunjungan: 40.000 jadwal kontrol per bulan.
- Dua pengingat: 80.000 eksekusi, 0,5 detik, 0,25 GB: 10.000 GB-detik.
- *Serverless* sekitar Rp3.000 per bulan tanpa kuota gratis; VM Rp650.000, pemanfaatan sekitar 1,5 persen.
- Biaya pesan WhatsApp Rp28 juta per bulan, apa pun *runtime*-nya.
- Argumen utama: tanpa *patch* server, kapasitas menyesuaikan sendiri.

Kasus Medikara: Hasil Laboratorium

- 32 berkas per hari, sekitar 150 hasil per berkas.
- Desain awal: satu fungsi terjadwal memproses semua berkas yang menunggu.
- Sistem laboratorium mati lima jam: 10 berkas, 20 menit, melampaui batas 15 menit.
- Perbaikan: fungsi pemecah per berkas, lalu satu hasil per eksekusi.
- Fungsi, antrean, dan penyimpanan di *region* Jakarta (matriks Bab 2).

PRADITA University

Memilih Runtime dan Vendor Lock-in

Kriteria Memilih Runtime

Kriteria	Pertanyaan kunci	Bobot
Lokasi data	Tersedia di lokasi yang diizinkan (Bab 2)?	Gugur
Kecocokan pola beban	Kapan dan seberapa sering dipakai?	25
Kebutuhan latensi	Adakah orang yang menunggu jawaban?	15
Keterampilan tim dan pasar	Siapa yang mengoperasikan pukul 02.00?	25
Biaya total tiga tahun	Termasuk tenaga kerja dan biaya keluar?	20
Portabilitas dan *lock-in*	Berapa upaya pindah penyedia?	15

Kriteria dan bobot ditulis sebelum opsi mana pun dinilai.

Matriks Keputusan: Layanan Janji Temu

Kriteria	Bobot	SaaS	PaaS	*Managed*	*Serverless*	VM
Lokasi data		Gugur	Lolos	Lolos	Lolos	Lolos
Pola beban	25		4	5	4	2
Latensi	15		4	5	3	4
Keterampilan tim	25		5	2	4	4
Biaya total	20		4	2	4	3
Portabilitas	15		3	4	2	4
Nilai berbobot		**-**	**4,10**	**3,50**	**3,55**	**3,30**

Keterampilan *managed* naik ke 4: nilainya 4,00. Pengingat kontrol: *serverless* 4,45, VM 3,00.

Vendor Lock-in dan Biaya Keluar

PRADITA University

- Pindah penyedia menjadi mahal karena teknologi, format data, atau kontrak.
- Biaya keluar: transfer data keluar, penulisan ulang, pelatihan, operasi paralel.
- Misalkan 40 fungsi, 25 dengan pemicu khas: sekitar 75 hari kerja.
- *Managed* Kubernetes: 12 dari 60 manifest khas, ditambah *registry* dan izin: sekitar 16 hari kerja.
- *Lock-in* boleh, asal disadari, dihitung, dan punya rencana keluar.

Aktivitas: Matriks Keputusan Runtime

PRADITA University

1. Ambil kandidat layanan dari Bab 6 dan Bab 7.
2. Tetapkan kriteria dan bobotnya sebelum menilai.
3. Terapkan kriteria gugur lokasi data, lalu nilai SaaS, PaaS, *managed* Kubernetes, *serverless*, dan VM.
4. Pilih *runtime* dan catat alasannya dalam ADR singkat.

Luaran: matriks keputusan *runtime* per layanan untuk C3.

Peran dan Berkas Pendukung

PRADITA University

- Anggota teknis: konsekuensi operasional setiap opsi.
- Anggota non-teknis: biaya, pasar tenaga kerja, risiko ketergantungan.
- Seluruh anggota: menyepakati bobot sebelum penilaian.
- `sources/chapter08/matriks-keputusan-runtime.xlsx`
- Opsional: `sources/chapter08/contoh-container/` di klaster kind.

Ringkasan

- *Container* membawa lingkungannya sendiri; bukan VM kecil.
- Kubernetes menjaga *desired state* lewat rekonsiliasi.
- Kubernetes memindahkan pekerjaan operasi, tidak menghapusnya.
- *Serverless* cocok untuk beban jarang, berkala, atau dipicu data.
- *Runtime* yang tepat sanggup dijalankan tim yang ada, dengan rencana keluar tertulis.

Latihan

PRADITA University

1. Jelaskan kepada direktur keuangan mengapa layanan pengingat jadwal kontrol pasien cocok dijalankan secara *serverless*.
2. Kapan Kubernetes justru menjadi beban bagi organisasi? Berikan dua kondisi beserta alasannya.
3. Bandingkan risiko *vendor lock-in* pada layanan *serverless* dan pada *managed* Kubernetes.